ADVANCED CS PRACTICUM | DISTRIBUTED BLOCKCHAIN TAKEAWAY ASSIGNMENT

MiniChain Network: Build, Synchronize, Attack, and Validate

Name: Yashank | **Roll Number:** t26049

1. System Architecture

MiniChain is a peer-to-peer replicated ledger run as three autonomous OS processes in separate terminals: **node1 (localhost:5001)**, **node2 (localhost:5002)** and **node3 (localhost:5003)**. The processes share no in-memory Python objects and no disk file: each keeps its own in-memory `Blockchain` and persists it only to its own JSON file, so a restarted node recovers its local history.

File (GitHub repo)	Role in the architecture
blockchain.py	`Block` (hashing, PoW mining, serialization) and `Blockchain` (Genesis, block and full-chain validation)
node.py	`BlockchainNode`: multi-threaded TCP server, peer messaging, mining, `sync_chain()`, interactive CLI
state_node1/2/3.json	Per-node persisted ledger
output/node1/2/3.txt	Per-node console logs (evidence in Section 4)

Block and Proof-of-Work (difficulty = 3). Each block is bound by one SHA-256 digest: **`hash = SHA256(index | timestamp | transactions | previous_hash | nonce | miner_id)`**, with transactions serialized as key-sorted JSON. A block is valid only if its hash starts with **000**. To make all three isolated nodes start from the same Genesis block B0 without shared files, `create_genesis_block()` uses fixed parameters (index=0, timestamp='0', transactions=['GENESIS'], previous_hash='0'*64, miner_id='system'), converging deterministically at nonce = 1199 with hash `000606778da36492f4abc3e4eb75f7a0b8d53989e3807c33e6e4a479c02c77d7`.

TCP protocol. Nodes exchange JSON messages with a 2.0 s socket timeout, so an offline peer never crashes an active node. **NEW_BLOCK** (node → peers) is broadcast after mining; the receiver checks `block.index == latest.index + 1`, the recomputed hash, `previous_hash == latest.hash` and the 000 prefix before appending. **GET_CHAIN** (node → peer) is issued by `sync` to every configured peer, and **CHAIN** (peer → node) returns the full chain, which the requester validates from B0 to tip with `validate_chain_list()` and adopts only if it is valid and longer (or repairs local corruption). The CLI offers: chain, mine, validate, peers, sync, tamper, exit.

2. Converged Blockchain Ledger State (B0 → B1 → B2 → B3)

Following the complete execution of propagation, tampering recovery, offline catch-up, and invalid-chain rejection tests (T1–T9), all three independent nodes (node1, node2, and node3) converged on the identical 4-block valid chain summarized below:

Block	Miner	Timestamp	Transactions	Nonce	Previous Block SHA-256 Hash	Mined Block SHA-256 Hash
B0	system	0	["GENESIS"]	1199	`000000000000000000000000...`	`000606778da36492f4abc3e4...`
B1	node1	1793531290	["A->B:10"]	14407	`000606778da36492f4abc3e4...`	`000e10228d9c5d5cbbb74395...`
B2	node2	1793531366	["B->C:5"]	10862	`000e10228d9c5d5cbbb74395...`	`000e40424f394d658980f6cc...`
B3	node1	1793531499	["C->D:3"]	930	`000e40424f394d658980f6cc...`	`00089b8ef98e0ccbf7d303b3...`

3. Test Case Verification Matrix (T1 – T9)

Test	Operation Executed	Expected Distributed Outcome	Actual Verified Result & Console Evidence
T1	Start node1 (5001), node2 (5002), and node3 (5003) in separate terminals.	Three independent node processes run; each begins with the same Genesis block B0.	**PASS:** All three nodes initialized on ports 5001–5003 with identical Genesis hash 000606778d... and logged Chain summary: B0(000606).
T2	On node1: mine A->B:10	node1 mines Block 1 and broadcasts it; node2 and node3 validate and append Block 1.	**PASS:** node1 mined B1 (000e10228d..., nonce 14407) and broadcast to node2, node3; both logged Hash valid, Previous hash valid, Proof-of-work valid, Block appended.
T3	On node2: mine B->C:5	Block 2 propagates; all three nodes converge on identical B0 -> B1 -> B2 chains.	**PASS:** node2 mined B2 (000e40424f..., nonce 10862) and broadcast to node1, node3; all three nodes converged on B0(000606) -> B1(000e10) -> B2(000e40).
T4	On node2: tamper 1 0 A->B:1000, then run validate across nodes.	node2 reports INVALID. node1 and node3 remain VALID and unchanged.	**PASS:** node2 logged Tampered Block 1 Tx 0: 'A->B:10' -> 'A->B:1000' and Local chain INVALID after tamper. node1 and node3 both logged Local chain VALID.
T5	On node2: restore/sync via sync and run validate.	node2 obtains a valid chain from peers and returns to the same chain as node1/node3.	**PASS:** node2 logged Local height: 2 (Valid: False), verified Peer (node1) height: 2 and Peer (node3) height: 2 as valid, logged Local chain replaced and Local chain VALID.
T6	Stop node3 (exit). On node1: mine C->D:3.	node1 and node2 reach B3; node3 remains one block behind at B2 while offline.	**PASS:** node1 mined B3 (00089b8ef9..., nonce 930), node2 appended B3, and node1 logged Peer node3 (localhost:5003) unreachable during broadcast.
T7	Restart node3 on port 5003 and run sync.	node3 fetches a longer valid chain from peers and reaches the same B3 tip.	**PASS:** node3 restarted at Local height: 2 (Valid: True), verified Peer (node1) height: 3 and Peer (node2) height: 3 (Candidate chain valid), and logged Local chain replaced -> B3(00089b).
T8	Present a longer invalid chain: node2 injects 2 invalid blocks (height 5); node1 runs sync.	The receiving node (node1) rejects the peer chain even though it is longer (height 5 > 3).	**PASS:** node1 (at Local height: 3) inspected Peer (node2) height: 5, logged Candidate chain from node2 INVALID - rejected, and logged Local chain retained.

Test	Operation Executed	Expected Distributed Outcome	Actual Verified Result & Console Evidence
T9	Present a longer valid chain: reset node1 to B0 (height 0) and run sync against height-3 peers.	The receiving node validates the candidate chain and adopts it under the longest-valid-chain rule.	**PASS:** node1 (at Local height: 0) validated height-3 chains from node2 and node3 (Candidate chain valid) and logged Local chain replaced.
After T7–T9 (Final Convergence)	B0 -> B1 -> B2 -> B3 (Height 3, VALID)	B0 -> B1 -> B2 -> B3 (Height 3, VALID)	B0 -> B1 -> B2 -> B3 (Height 3, VALID)

4. Recorded Console Evidence (output/node1.txt, output/node2.txt, output/node3.txt)

output/node1.txt (Port 5001)

```
[node1] Started on port 5001 with
Genesis 000606778d...
[node1] Mining Block 1...
[node1] Block mined: 000e10228d...
[node1] Broadcasting Block 1 to node2,
node3
[node1] Block 2 received from node2
[node1] Hash valid | Prev valid | PoW
valid
[node1] Block appended
[node1] Chain:
B0(000606)->B1(000e10)->B2(000e40)
[node1] Local chain VALID
[node1] Mining Block 3...
[node1] Block mined: 00089b8ef9...
[node1] Peer node3 (localhost:5003)
unreachable
[node1] Local height: 3 (Valid: True)
[node1] Peer (node2) height: 5
[node1] Candidate chain from node2
INVALID - rejected
[node1] Local height: 0 (Valid: True)
[node1] Peer (node2) height: 3 ->
Candidate valid
[node1] Local chain replaced
```

output/node2.txt (Port 5002)

```
[node2] Started on port 5002 with
Genesis 000606778d...
[node2] Block 1 received from node1
[node2] Hash valid | Prev valid | PoW
valid
[node2] Block appended
[node2] Mining Block 2...
[node2] Block mined: 000e40424f...
[node2] Broadcasting Block 2 to node1,
node3
[node2] Tampered Block 1 Tx 0:
'A->B:10'->'A->B:1000'
[node2] Local chain INVALID after tamper
[node2] Local height: 2 (Valid: False)
[node2] Peer (node1) height: 2 ->
Candidate valid
[node2] Local chain replaced | Local
chain VALID
[node2] Block 3 received from node1 ->
Appended
[node2] Injected 2 invalid blocks for T8
(Height: 5)
[node2] Local height: 5 (Valid: False)
[node2] Peer (node1) height: 3 ->
Candidate valid
[node2] Local chain replaced
[node2] Chain: B0->B1->B2->B3(00089b)
```

output/node3.txt (Port 5003)

```
[node3] Started on port 5003 with
Genesis 000606778d...
[node3] Chain summary: B0(000606)
[node3] Block 1 received from node1
[node3] Hash valid | Prev valid | PoW
valid
[node3] Block appended
[node3] Block 2 received from node2
[node3] Hash valid | Prev valid | PoW
valid
[node3] Block appended
[node3] Chain:
B0(000606)->B1(000e10)->B2(000e40)
[node3] Local chain VALID
[node3] Stopping node...
[node3] Started on port 5003 with
Genesis 000606778d...
[node3] Local height: 2 (Valid: True)
[node3] Peer (node1) height: 3
[node3] Candidate chain valid
[node3] Peer (node2) height: 3 ->
Candidate valid
[node3] Local chain replaced
[node3] Chain: B0->B1->B2->B3(00089b)
```

5. Technical Analysis & Distributed Consensus Evaluation

Q1. How does MiniChain detect tampering, and why is the damage contained? (T4 & T5)

In T4, `tamper 1 0 A->B:1000` changed Block 1 on node2 without re-mining. `validate_chain_list()` recomputed B1's SHA-256 and found it matched neither the stored hash (000e10228d9c5d5c...) nor Block 2's `previous_hash`. Even a recomputed hash would lose the 000 prefix unless every later block were re-mined faster than the honest network. node1 and node3 run in separate processes and memory, so they stayed VALID. In T5, `sync` on node2 fetched the height-2 chains via GET_CHAIN, verified them from B0 to B2, and replaced the corrupted ledger.

Q2. Why must consensus use the longest *valid* chain rather than just the longest chain? (T8 & T9)

A rule that trusts the highest block count is open to forgery, because a malicious node can append unmined blocks in microseconds. In T8, node2 forged two blocks to reach height 5 against honest height 3. node1 ran `validate_chain_list(candidate, difficulty=3)` before comparing lengths; the forged hashes failed the recomputation and the 000 target, so node1 logged 'Candidate chain from node2 INVALID - rejected' and kept its chain. In T9, node1 was reset to B0 and a fully verified height-3 chain was adopted under: `len(peer) > len(local) and valid(peer)`.

Q3. How does the network tolerate a crashed node and catch it up? (T6 & T7)

In T6, node3 exited at height 2. When node1 mined B3 (nonce 930), `broadcast_block()` reached node2 and caught the refused connection to localhost:5003 under the 2.0 s timeout, logging 'unreachable' without any unhandled exception. In T7, node3 reloaded height 2 from state_node3.json, queried peers via sync, verified the height-3 candidate against the identical Genesis and all PoW links, and updated its ledger to B3.

Q4. Is Proof-of-Work difficulty 3 meaningful for this demo?

Each hash starts with three hex zeroes with probability $1/16^3 = 1/4096$, so about 4096 attempts are expected per block. The recorded nonces (1199, 14407, 10862, 930) are consistent with that scale. This is cheap for an honest miner but makes rewriting a tampered history costly, because every later block would need re-mining.